

Java Collection Framework : Important methods by Interface

1 . Title: Demonstration of Important Methods of Collection Interface in Java Aim /

Objective : To write and execute a Java program to demonstrate the important methods of the “Collection” interface such as “add()”, “addAll()”, “remove()”, “removeAll()”, “contains()”, “containsAll()”, “size()”, “isEmpty()”, “clear()”, and “iterator()”.

Steps :

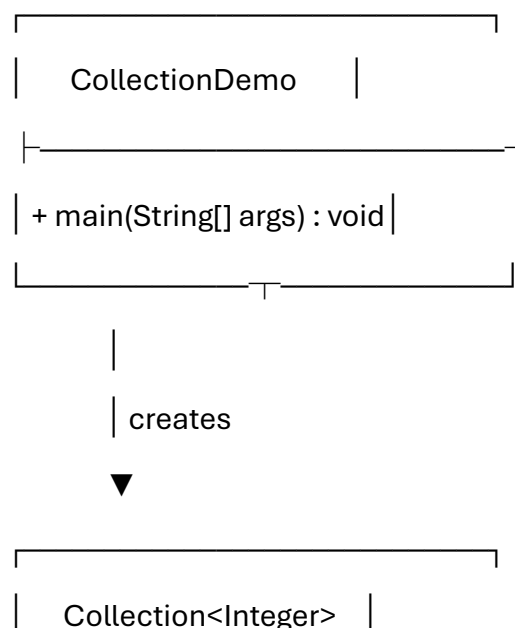
1. Start the program.
2. Import ArrayList, Collection, and Iterator.
3. Create two Collection<Integer> objects named c1 and c2 using ArrayList.
4. Add 10, 20, and 30 to collection c1.
5. Add 20 and 30 to collection c2.
6. Display both collections.
7. Use addAll() to add all elements of c2 into c1.
8. Use contains() to check whether 20 is present in c1.
9. Use containsAll() to check whether all elements of c2 are present in c1.
10. Use size() to find the number of elements in c1.
11. Use isEmpty() to check whether c1 is empty.
12. Use remove() to remove 10 from c1.
13. Use removeAll() to remove all elements of c2 from c1.
14. Add 50 and 60 to c1.
15. Create an Iterator and traverse the elements of c1.
16. Use clear() to remove all elements from c1.
17. Display the empty collection.
18. Stop the program.

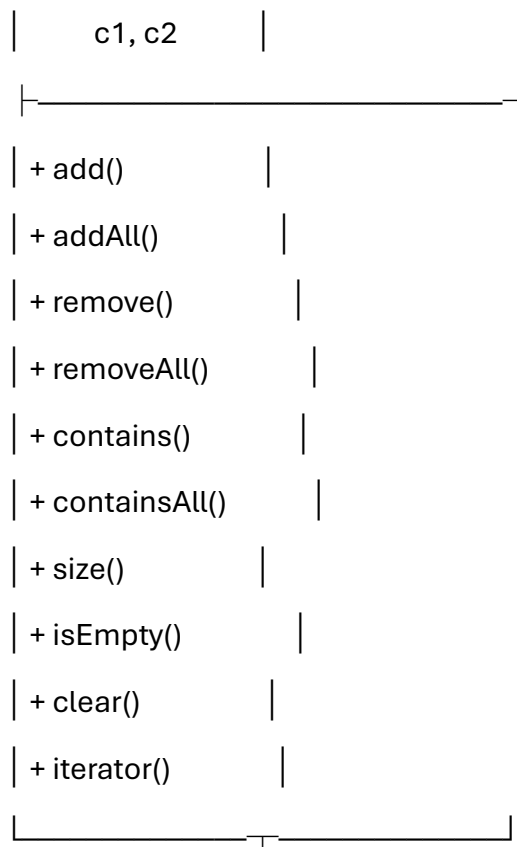
Algorithm: Collection Interface

1. Start the program.
2. Import ArrayList, Collection, and Iterator classes.
3. Create two collection objects c1 and c2 using ArrayList.

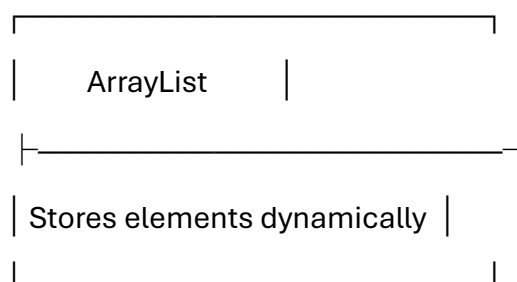
4. Add the elements 10, 20, and 30 to c1.
5. Add the elements 20 and 30 to c2.
6. Display the elements of both collections.
7. Use addAll() to add all elements of c2 to c1.
8. Use contains() to check whether 20 is present in c1.
9. Use containsAll() to check whether all elements of c2 are present in c1.
10. Use size() to find the number of elements in c1.
11. Use isEmpty() to check whether c1 is empty.
12. Use remove() to remove the element 10 from c1.
13. Use removeAll() to remove all elements of c2 from c1.
14. Add 50 and 60 to c1.
15. Create an Iterator object for c1.
16. Use hasNext() and next() to traverse and display the elements of c1.
17. Use clear() to remove all elements from c1.
18. Display the final collection.
19. Stop the program.

Uml Diagram :

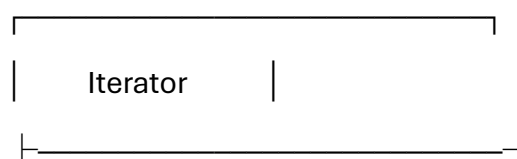




implemented by
▼



iterator()
▼



```
| + hasNext() : boolean |  
| + next() : Integer    |
```

Code :

```
import java.util.ArrayList;  
  
import java.util.Collection;  
  
import java.util.Iterator;  
  
public class CollectionDemo {  
  
    public static void main(String[] args) {  
  
        Collection<Integer> c1 = new ArrayList<>();  
  
        Collection<Integer> c2 = new ArrayList<>();  
  
        // add()  
  
        c1.add(10);  
  
        c1.add(20);  
  
        c1.add(30);  
  
        c2.add(20);  
  
        c2.add(30);  
  
        System.out.println("Collection 1: " + c1);  
  
        System.out.println("Collection 2: " + c2);  
  
        // addAll()  
  
        c1.addAll(c2);  
  
        System.out.println("After addAll(): " + c1);  
  
        // contains()  
  
        System.out.println("Contains 20: " + c1.contains(20));  
  
        // containsAll()  
  
        System.out.println("Contains all c2: " + c1.containsAll(c2));  
  
        // size()  
  
        System.out.println("Size: " + c1.size());  
    }  
}
```

```
// isEmpty()
System.out.println("Is Empty: " + c1.isEmpty());

// remove()
c1.remove(10);

System.out.println("After remove(10): " + c1);

// removeAll()
c1.removeAll(c2);

System.out.println("After removeAll(c2):" + c1);

// Add elements again
c1.add(50);
c1.add(60);

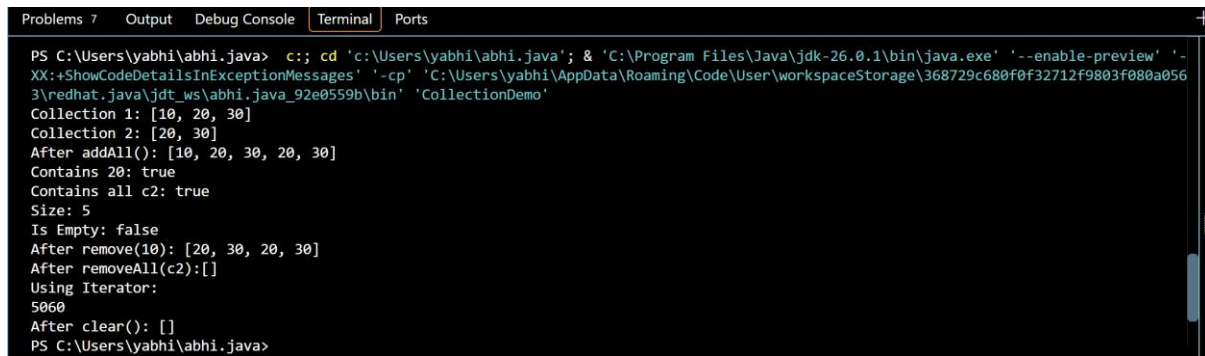
// iterator()
System.out.println("Using Iterator: ");
Iterator<Integer> itr = c1.iterator();
while (itr.hasNext()) {
    System.out.print(itr.next() + "");
}

System.out.println();

// clear()
c1.clear();

System.out.println("After clear(): " + c1);
}
}
```

Output :



```
PS C:\Users\yabhi\abhi.java> c:: cd 'c:\Users\yabhi\abhi.java'; & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\yabhi\AppData\Roaming\Code\User\workspaceStorage\368729c680f0f32712f9803f080a0563\redhat.java\jdt_ws\abhi.java_92e0559b\bin' 'CollectionDemo'
Collection 1: [10, 20, 30]
Collection 2: [20, 30]
After addAll(): [10, 20, 30, 20, 30]
Contains 20: true
Contains all c2: true
Size: 5
Is Empty: false
After remove(10): [20, 30, 20, 30]
After removeAll(c2): []
Using Iterator:
5060
After clear(): []
PS C:\Users\yabhi\abhi.java>
```

2. Title : Demonstration of Important Methods of List Interface in Java.

Aim / Objective : To write and execute a Java program to demonstrate the important methods of the “List” interface.

System Requirements :

Hardware: Computer/Laptop, minimum 4 GB RAM, Keyboard and Mouse.

Software: JDK, JRE, Java-supported IDE such as VS Code/Eclipse, and Windows/Linux/macOS.

Steps

1. Start the program.
2. Import ArrayList, Comparator, and List.
3. Create a List<Integer> using ArrayList.
4. Add the elements 30, 10, 20, and 10 to the list.
5. Display the list.
6. Insert 15 at index 1 using add(index, element).
7. Display the element at index 2 using get().
8. Replace the element at index 2 with 25 using set().
9. Remove the element at index 1 using remove().
10. Find the first occurrence of 10 using indexOf().
11. Find the last occurrence of 10 using lastIndexOf().
12. Display the elements from index 1 to 2 using subList().

13. Sort the list in natural ascending order using sort().

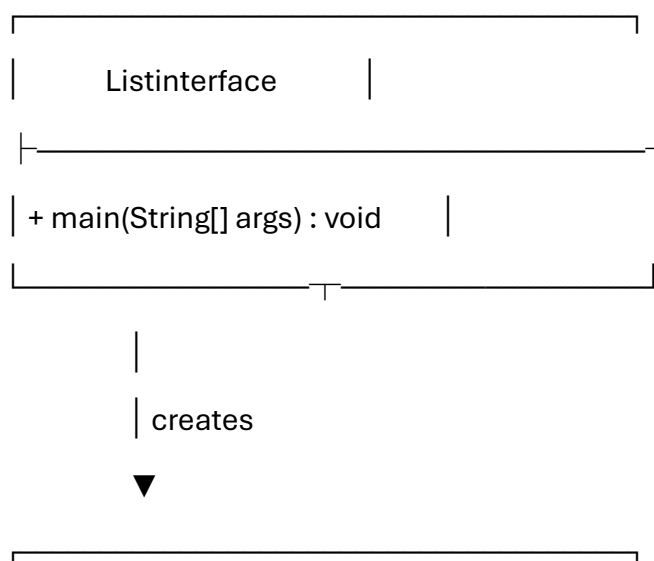
14. Display the sorted list.

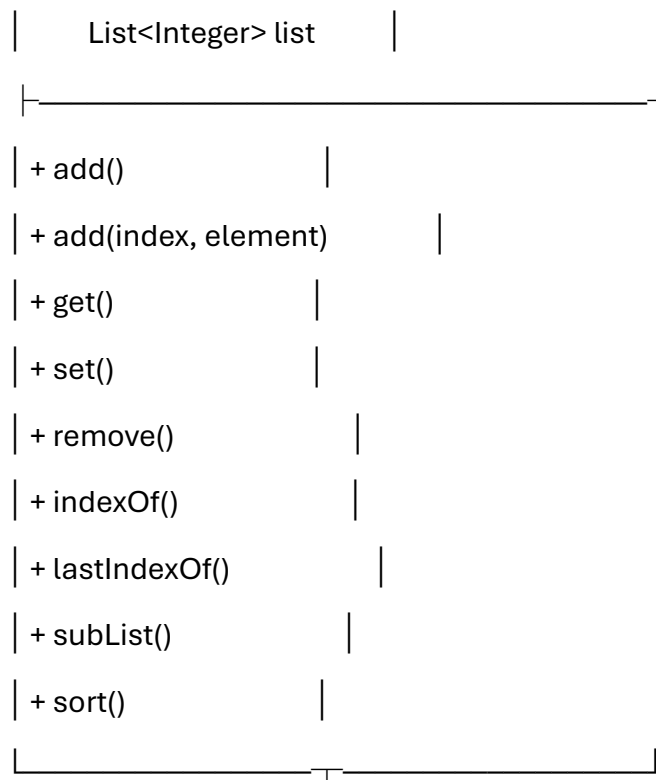
15. Stop the program.

Algorithm

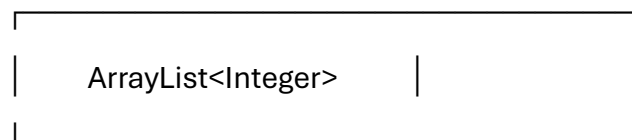
1. Start.
2. Create an empty integer list using ArrayList.
3. Add 30, 10, 20, and 10 to the list.
4. Display the list.
5. Insert 15 at index 1.
6. Retrieve and display the element at index 2.
7. Replace the element at index 2 with 25.
8. Remove the element at index 1.
9. Find and display the first index of 10.
10. Find and display the last index of 10.
11. Create and display a sublist from index 1 to index 3.
12. Sort the list in ascending order using Comparator.naturalOrder().
13. Display the sorted list.
14. Stop.

UML Diagram

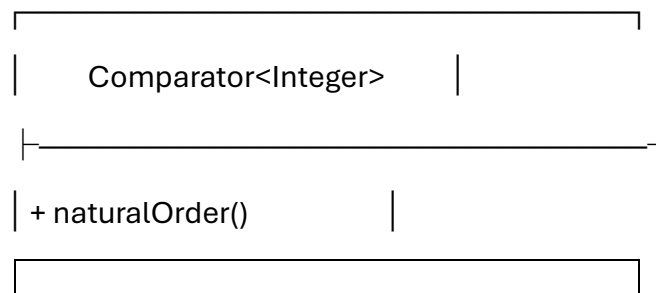




implemented by



uses



Code : `import java.util.ArrayList;`


```
import java.util.Comparator;

import java.util.List;

public class Listinterface{

    public static void main(String[] args) {

        // Create a List using ArrayList

        List<Integer> list = new ArrayList<>();

        list.add(30);

        list.add(10);

        list.add(20);

        list.add(10);

        System.out.println("List: " + list);

        // Insert an element at index 1

        list.add(1, 15);

        System.out.println("After add(1,15): " + list);

        System.out.println("Element at index 2: " + list.get(2));

        list.set(2, 25);

        System.out.println("After set(): " + list);

        list.remove(1);

        System.out.println("After remove(): " + list);
```

```

        System.out.println("First index of 10: " + list.indexOf(10));

        System.out.println("Last index of 10: " + list.lastIndexOf(10));


        System.out.println("SubList: " + list.subList(1, 3));


        list.sort(Comparator.naturalOrder());

        System.out.println("After sort(): " + list);
    }
}

```

Output :

```

Problems 8  Output  Debug Console  Terminal  Ports

PS C:\Users\yabhi\abhi.java> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '-e
ges' '-cp' 'C:\Users\yabhi\AppData\Roaming\Code\User\workspaceStorage\368729c680f0f3
559b\bin' 'ListInterface'
List: [30, 10, 20, 10]
After add(1,15): [30, 15, 10, 20, 10]
Element at index 2: 10
After set(): [30, 15, 25, 20, 10]
After remove(): [30, 25, 20, 10]
First index of 10: 3
Last index of 10: 3
SubList: [25, 20]
After sort(): [10, 20, 25, 30]
PS C:\Users\yabhi\abhi.java>

```

3. Title : Demonstration of Important Methods of Set Interface in Java.

Aim / Objective : To write and execute a Java program to demonstrate the important methods of the “Set” interface.

System Requirements :

Hardware: Computer/Laptop, minimum 4 GB RAM.

Software: JDK, JRE, Java-supported IDE, and suitable operating system.

Steps :

1. Start the program.

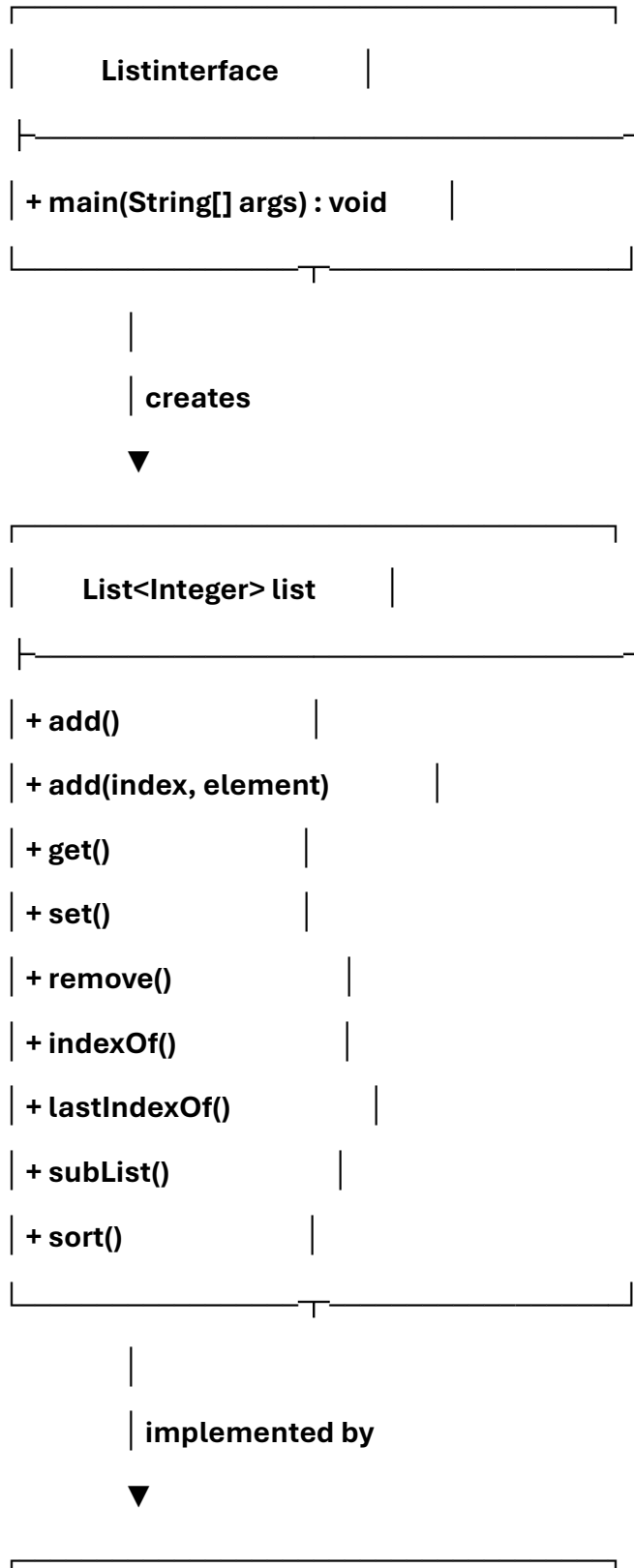
2. Import ArrayList, Comparator, and List.
3. Create a List<Integer> using ArrayList.
4. Add the elements 30, 10, 20, and 10 to the list.
5. Display the list.
6. Insert 15 at index 1 using add(index, element).
7. Display the element at index 2 using get().
8. Replace the element at index 2 with 25 using set().
9. Remove the element at index 1 using remove().
10. Find the first occurrence of 10 using indexOf().
11. Find the last occurrence of 10 using lastIndexOf().
12. Display the elements from index 1 to 2 using subList().
13. Sort the list in natural ascending order using sort().
14. Display the sorted list.
15. Stop the program.

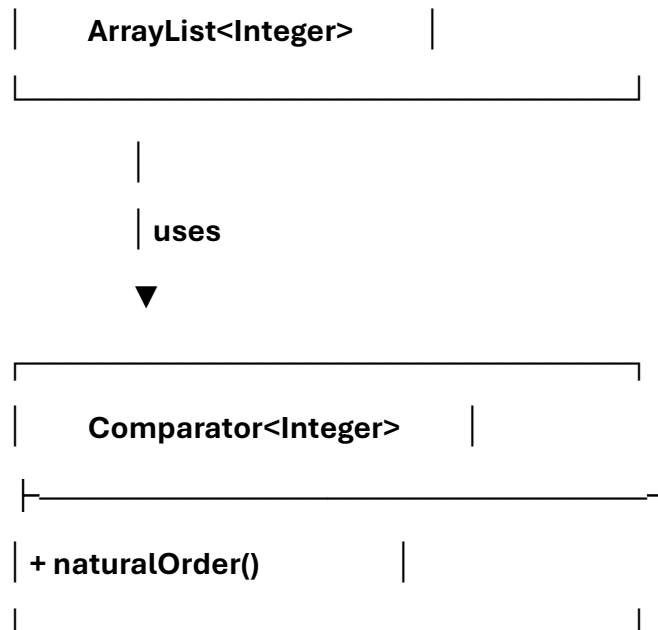
Algorithm :

1. Start.
2. Create an empty integer list using ArrayList.
3. Add 30, 10, 20, and 10 to the list.
4. Display the list.
5. Insert 15 at index 1.
6. Retrieve and display the element at index 2.
7. Replace the element at index 2 with 25.
8. Remove the element at index 1.
9. Find and display the first index of 10.
10. Find and display the last index of 10.
11. Create and display a sublist from index 1 to index 3.
12. Sort the list in ascending order using Comparator.naturalOrder().
13. Display the sorted list.

14. Stop.

UML Diagram :





Code :

```
import java.util.HashSet;
import java.util.Set;

public class SetDemo {
    public static void main(String[] args) {

        // Create a Set using HashSet
        Set<Integer> set = new HashSet<>();

        set.add(10);
        set.add(20);
        set.add(30);
        set.add(20);

        System.out.println("Set: " + set);

        // Duplicate values are not stored in a Set
```

```

System.out.println("Contains 20: " + set.contains(20));

System.out.println("Size: " + set.size());

System.out.println("Is Empty: " + set.isEmpty());


set.remove(10);

System.out.println("After remove(10): " + set);


set.clear();

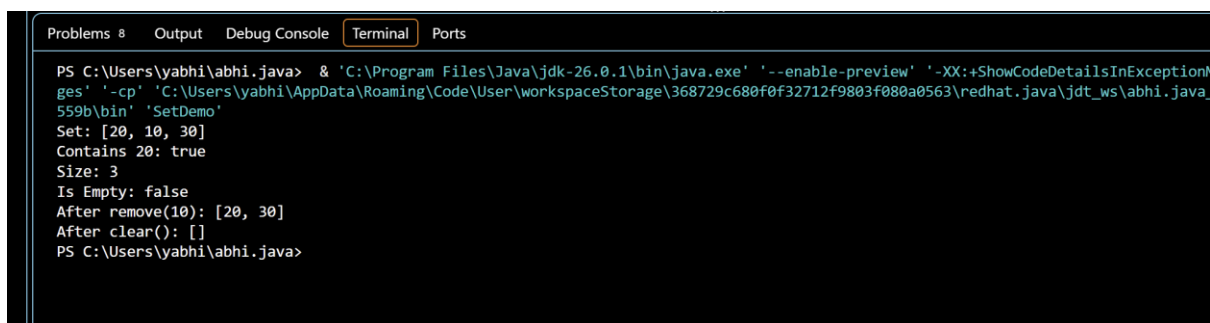
System.out.println("After clear(): " + set);

}

}

```

Output :



```

PS C:\Users\yabhi\abhi.java> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\yabhi\AppData\Roaming\Code\User\workspaceStorage\368729c680f0f32712f9803f080a0563\redhat.java\jdt_ws\abhi.java_559b\bin' 'SetDemo'
Set: [20, 10, 30]
Contains 20: true
Size: 3
Is Empty: false
After remove(10): [20, 30]
After clear(): []
PS C:\Users\yabhi\abhi.java>

```

4. Title :

Demonstration of Important Methods of SortedSet Interface in Java

Aim / Objective : To write and execute a Java program to demonstrate the important methods of the “SortedSet” interface. **System Requirements :**

Hardware: Computer/Laptop, minimum 4 GB RAM.

Software: JDK, JRE, Java IDE, and operating system.

Steps :

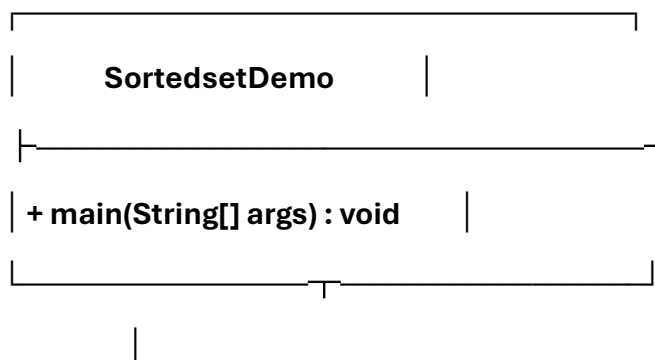
1. Start the program.
2. Import SortedSet and TreeSet.
3. Create a SortedSet<Integer> using TreeSet.

4. Add the elements 40, 10, 30, and 20 to the set.
5. Display the sorted set.
6. Use first() to find and display the smallest element.
7. Use last() to find and display the largest element.
8. Use headSet(30) to display elements less than 30.
9. Use tailSet(20) to display elements greater than or equal to 20.
10. Use subSet(20, 40) to display elements from 20 inclusive to 40 exclusive.
11. Use comparator() to display the comparator used for ordering.
12. Stop the program.

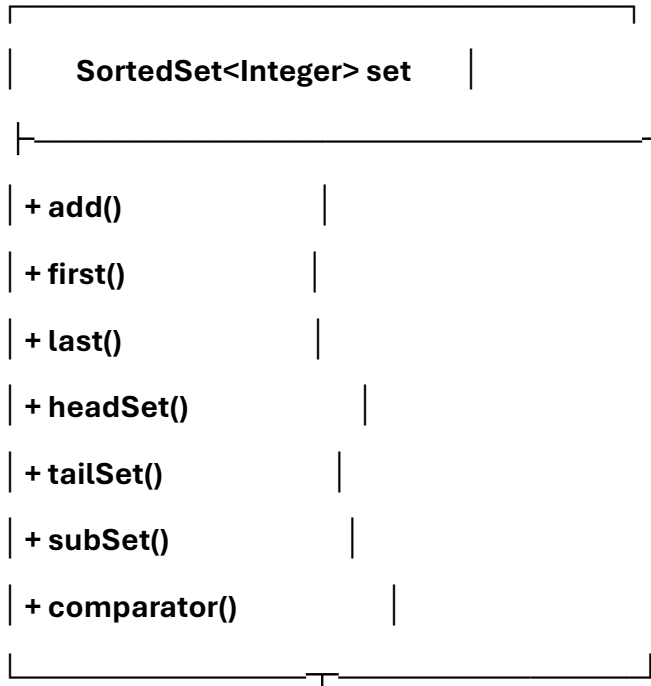
Algorithm :

1. Start.
2. Create an empty SortedSet using TreeSet.
3. Add 40, 10, 30, and 20 to the set.
4. Display the elements of the sorted set.
5. Find and display the first element using first().
6. Find and display the last element using last().
7. Display elements less than 30 using headSet(30).
8. Display elements greater than or equal to 20 using tailSet(20).
9. Display elements from 20 inclusive to 40 exclusive using subSet(20, 40).
10. Display the comparator using comparator().
11. Stop.

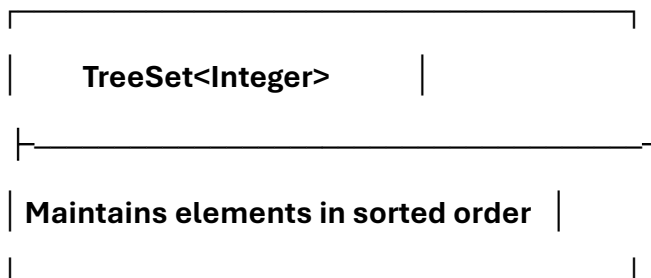
UML Diagram :



| creates



|
| implemented by



Code :

```
import java.util.SortedSet;
```

```
import java.util.TreeSet;
```

```
public class SortedsetDemo {
```

```
    public static void main(String[] args) {
```



```

// TreeSet maintains elements in sorted order

SortedSet<Integer> set = new TreeSet<>();


set.add(40);

set.add(10);

set.add(30);

set.add(20);


System.out.println("Sorted Set: " + set);


System.out.println("First: " + set.first());

System.out.println("Last: " + set.last());


// Display different portions of the sorted set

System.out.println("HeadSet(30): " + set.headSet(30));

System.out.println("TailSet(20): " + set.tailSet(20));

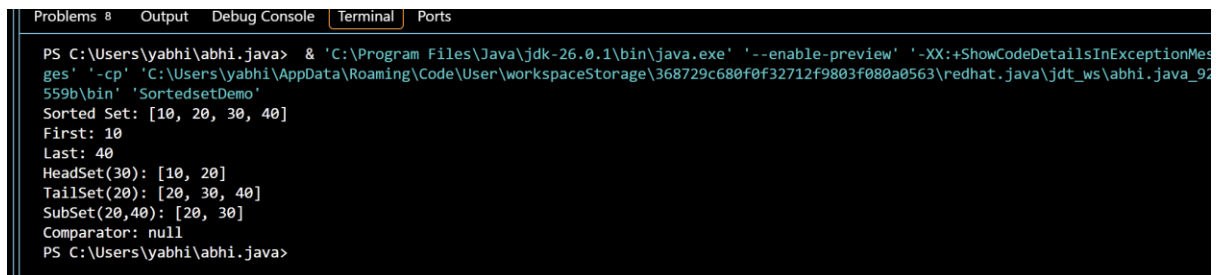
System.out.println("SubSet(20,40): " + set.subSet(20, 40));


System.out.println("Comparator: " + set.comparator());

}

}

```



```

Problems 8  Output  Debug Console  Terminal  Ports
PS C:\Users\yabhi\abhi.java> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\yabhi\AppData\Roaming\Code\User\workspaceStorage\368729c680f0f32712f9803f080a0563\redhat.java\jdt_ws\abhi.java_92559b\bin' 'SortedSetDemo'
Sorted Set: [10, 20, 30, 40]
First: 10
Last: 40
HeadSet(30): [10, 20]
TailSet(20): [20, 30, 40]
SubSet(20,40): [20, 30]
Comparator: null
PS C:\Users\yabhi\abhi.java>

```

5.Title :. Demonstration of Important Methods of NavigableSet Interface in Java.

Aim / Objective :. To demonstrate navigation and removal methods of the “NavigableSet” interface.

System Requirements :

Hardware: Computer/Laptop, minimum 4 GB RAM.

Software: JDK, JRE, Java IDE, and operating system.

Steps :

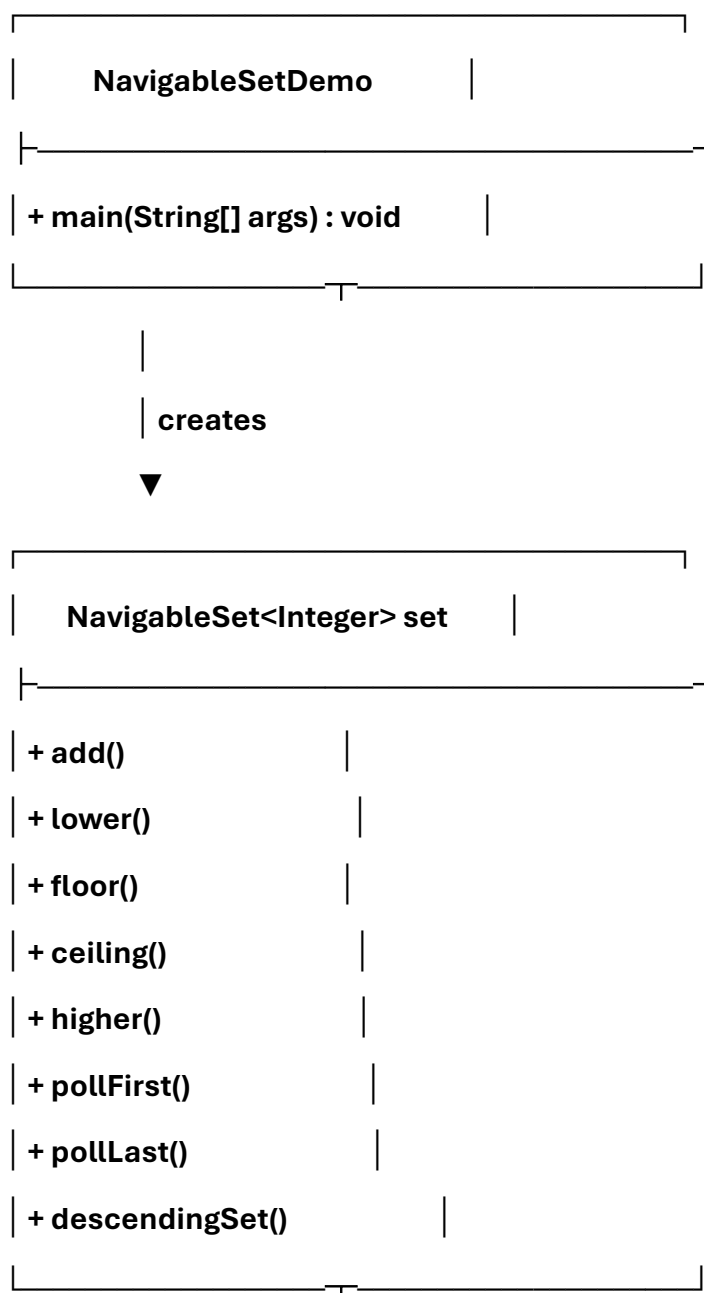
1. Start the program.
2. Import NavigableSet and TreeSet.
3. Create a NavigableSet<Integer> using TreeSet.
4. Add the elements 10, 20, 30, and 40 to the set.
5. Display the set.
6. Use lower(30) to find the greatest element less than 30.
7. Use floor(30) to find the greatest element less than or equal to 30.
8. Use ceiling(25) to find the smallest element greater than or equal to 25.
9. Use higher(30) to find the smallest element greater than 30.
10. Use pollFirst() to remove and display the first element.
11. Use pollLast() to remove and display the last element.
12. Display the set after removing the first and last elements.
13. Use descendingSet() to display the elements in descending order.
14. Stop the program.

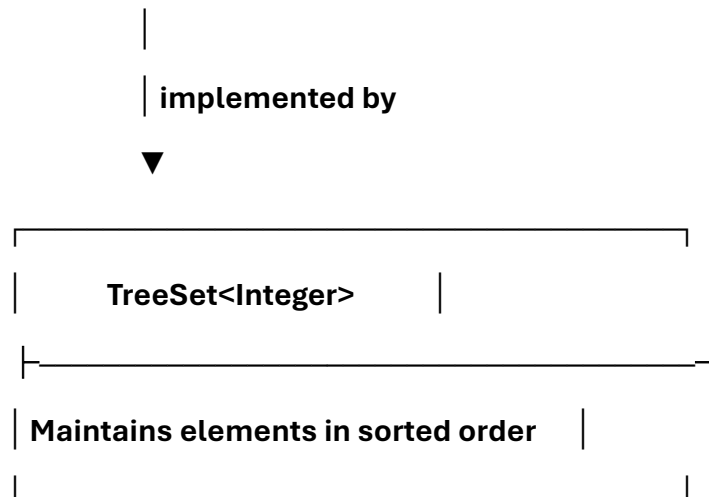
Algorithm :

1. Start.
2. Create an empty NavigableSet using TreeSet.
3. Add 10, 20, 30, and 40 to the set.
4. Display the set.
5. Find and display the element lower than 30 using lower().
6. Find and display the element lower than or equal to 30 using floor().

7. Find and display the element greater than or equal to 25 using ceiling().
8. Find and display the element higher than 30 using higher().
9. Remove and display the first element using pollFirst().
10. Remove and display the last element using pollLast().
11. Display the updated set.
12. Display the set in descending order using descendingSet().
13. Stop.

UML Diagram :





Code :

```
import java.util.NavigableSet;
import java.util.TreeSet;

public class NavigableSetDemo {

    // Create a NavigableSet using TreeSet
    NavigableSet<Integer> set = new TreeSet<>();

    set.add(10);
    set.add(20);
    set.add(30);
    set.add(40);

    System.out.println("Set: " + set);

    System.out.println("Lower(30): " + set.lower(30));
    System.out.println("Floor(30): " + set.floor(30));
```

```

System.out.println("Ceiling(25): " + set.ceiling(25));

System.out.println("Higher(30): " + set.higher(30));


// Remove the first and last elements

System.out.println("PollFirst(): " + set.pollFirst());

System.out.println("PollLast(): " + set.pollLast());


System.out.println("Set after polling: " + set);


System.out.println("Descending Set: " + set.descendingSet());

}

}

```

Output :

```

PS C:\Users\yabhi\abhi.java> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\yabhi\AppData\Roaming\Code\User\workspaceStorage\368729c680f0f32712f9803f080a0563\redhat.java\jdt_ws\abhi.java_92e0559b\bin' 'NavigableSetDemo'
Set: [10, 20, 30, 40]
Lower(30): 20
Floor(30): 30
Ceiling(25): 30
Higher(30): 40
PollFirst(): 10
PollLast(): 40
Set after polling: [20, 30]
Descending Set: [30, 20]
PS C:\Users\yabhi\abhi.java>

```

6.Title :

Demonstration of Important Methods of Queue Interface in Java.

Aim / Objective : To demonstrate the insertion, deletion, and retrieval methods of the “Queue” interface.

System Requirements :

Hardware: Computer/Laptop, minimum 4 GB RAM.

Software: JDK, JRE, Java IDE, and operating system.

Steps

1. Start the program.
2. Import LinkedList and Queue.

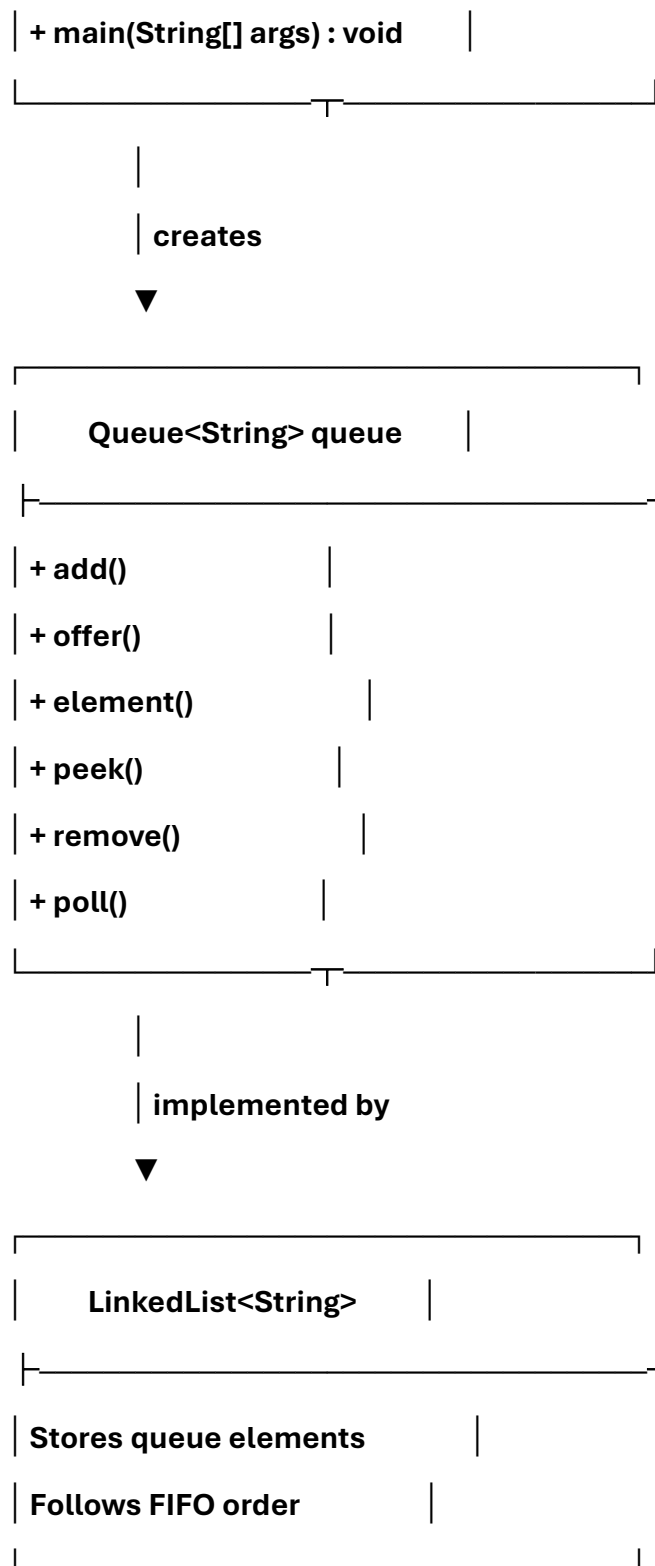
3. Create a Queue<String> using LinkedList.
4. Add "A" and "B" using add().
5. Add "C" using offer().
6. Display the queue.
7. Use element() to retrieve the front element.
8. Use peek() to retrieve the front element without removing it.
9. Use remove() to remove the front element.
10. Display the queue after removal.
11. Use poll() to remove the next front element.
12. Display the queue after polling.
13. Stop the program.

Algorithm

1. Start.
2. Create an empty queue using LinkedList.
3. Add elements A and B using add().
4. Add element C using offer().
5. Display the queue.
6. Retrieve and display the front element using element().
7. Retrieve and display the front element using peek() without removing it.
8. Remove and display the front element using remove().
9. Display the updated queue.
10. Remove and display the front element using poll().
11. Display the updated queue.
12. Stop.

UML Diagram





Code :

```
import java.util.LinkedList;
import java.util.Queue;
```

```

public class QueueDemo {

    public static void main(String[] args) {

        // Create a Queue using LinkedList

        Queue<String> queue = new LinkedList<>();

        queue.add("A");

        queue.add("B");

        queue.offer("C");


        System.out.println("Queue: " + queue);


        System.out.println("Element(): " + queue.element());

        System.out.println("Peek(): " + queue.peek());


        // Remove elements from the front of the queue

        System.out.println("Remove(): " + queue.remove());

        System.out.println("After remove(): " + queue);


        System.out.println("Poll(): " + queue.poll());

        System.out.println("After poll(): " + queue);

    }

}

```

Output :

```

PS C:\Users\yabhi\abhi.java> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInException' '-cp' 'C:\Users\yabhi\AppData\Roaming\Code\User\workspaceStorage\368729c680f0f32712f9803f080a0563\redhat.java\jdt_ws\abhi.559b\bin' 'QueueDemo'
Queue: [A, B, C]
Element(): A
Peek(): A
Remove(): A
After remove(): [B, C]
Poll(): B
After poll(): [C]
PS C:\Users\yabhi\abhi.java>

```


7.Title : Demonstration of Important Methods of Deque Interface in Java .

Aim / Objective : To demonstrate insertion, deletion, and retrieval operations at both ends of a “Deque”. **System Requirements :**

Hardware: Computer/Laptop, minimum 4 GB RAM.

Software: JDK, JRE, Java IDE, and operating system.

Steps :

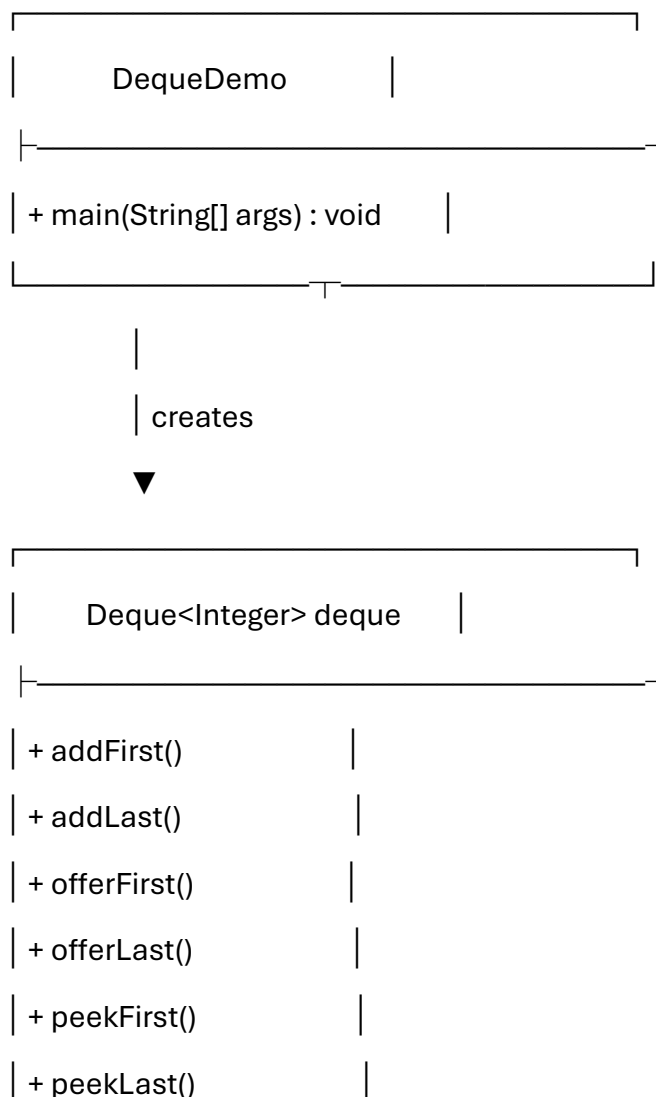
1. Start the program.
2. Import ArrayDeque and Deque.
3. Create a Deque<Integer> using ArrayDeque.
4. Add 20 at the front using addFirst().
5. Add 30 at the rear using addLast().
6. Add 10 at the front using offerFirst().
7. Add 40 at the rear using offerLast().
8. Display the deque.
9. Use peekFirst() to view the first element.
10. Use peekLast() to view the last element.
11. Use removeFirst() to remove the first element.
12. Use removeLast() to remove the last element.
13. Display the deque after removal.
14. Use pollFirst() to remove the first element.
15. Use pollLast() to remove the last element.
16. Display the final deque.
17. Stop the program.

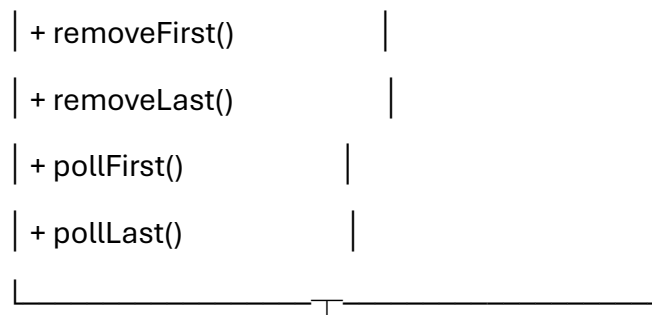
Algorithm :

1. Start.
2. Create an empty Deque using ArrayDeque.
3. Insert 20 at the front and 30 at the rear.
4. Insert 10 at the front and 40 at the rear.

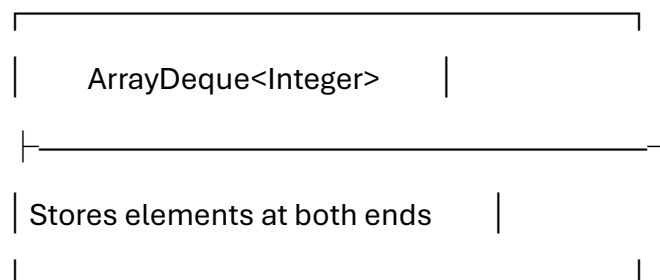
5. Display the deque.
6. Retrieve and display the first element using peekFirst().
7. Retrieve and display the last element using peekLast().
8. Remove and display the first element using removeFirst().
9. Remove and display the last element using removeLast().
10. Display the updated deque.
11. Remove and display the first element using pollFirst().
12. Remove and display the last element using pollLast().
13. Display the final deque.
14. Stop.

UML Diagram :





implemented by



Code :

```
import java.util.ArrayDeque;
import java.util.Deque;

public class DequeDemo {
    public static void main(String[] args) {

        // Create a Deque using ArrayDeque
        Deque<Integer> deque = new ArrayDeque<>();

        deque.addFirst(20);
        deque.addLast(30);

        deque.offerFirst(10);
        deque.offerLast(40);
```

```
System.out.println("Deque: " + deque);
```

```
System.out.println("PeekFirst(): " + deque.peekFirst());
```

```
System.out.println("PeekLast(): " + deque.peekLast());
```

```
// Remove elements from both ends
```

```
System.out.println("RemoveFirst(): " + deque.removeFirst());
```

```
System.out.println("RemoveLast(): " + deque.removeLast());
```

```
System.out.println("After remove operations: " + deque);
```

```
System.out.println("PollFirst(): " + deque.pollFirst());
```

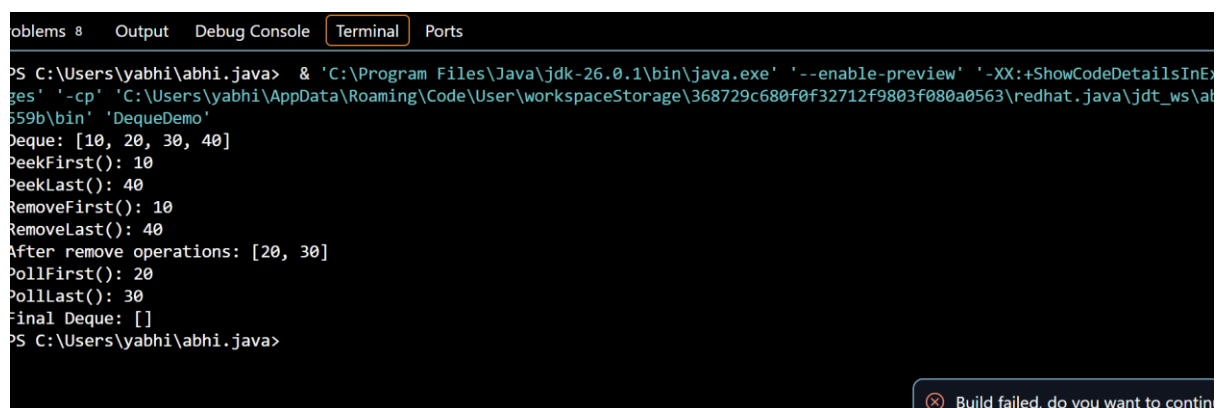
```
System.out.println("PollLast(): " + deque.pollLast());
```

```
System.out.println("Final Deque: " + deque);
```

```
}
```

```
}
```

Output :



```
PS C:\Users\yabhi\abhi.java> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\yabhi\AppData\Roaming\Code\User\workspaceStorage\368729c680f0f32712f9803f080a0563\redhat.java\jdt_ws\ab559b\bin' 'DequeDemo'
Deque: [10, 20, 30, 40]
PeekFirst(): 10
PeekLast(): 40
RemoveFirst(): 10
RemoveLast(): 40
After remove operations: [20, 30]
PollFirst(): 20
PollLast(): 30
Final Deque: []
PS C:\Users\yabhi\abhi.java>
```

8.Title :

Demonstration of Important Methods of Map Interface in Java.

Aim / Objective : To demonstrate the important methods of the “Map” interface for storing key-value pairs. **System Requirements :**

Hardware: Computer/Laptop, minimum 4 GB RAM.

Software: JDK, JRE, Java IDE, and operating system.

Steps – Map Interface

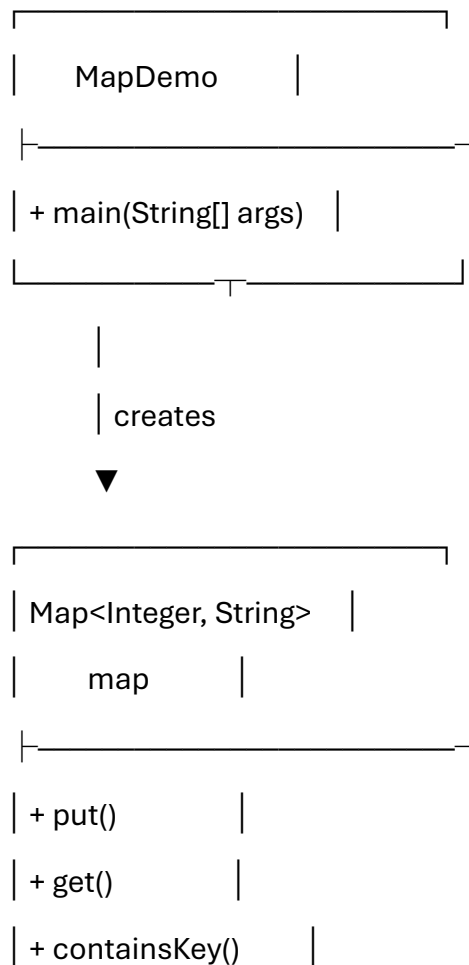
1. Start the program.
2. Import HashMap and Map from java.util.
3. Create a Map<Integer, String> using HashMap.
4. Add three key-value pairs using put().
5. Display the complete map.
6. Retrieve the value for key 102 using get().
7. Check whether key 101 exists using containsKey().
8. Check whether value "Bob" exists using containsValue().
9. Display all keys using keySet().
10. Display all values using values().
11. Display all key-value pairs using entrySet().
12. Display the size of the map using size().
13. Check whether the map is empty using isEmpty().
14. Remove the entry with key 103 using remove().
15. Clear all entries using clear().
16. Display the final empty map.
17. Stop the program.

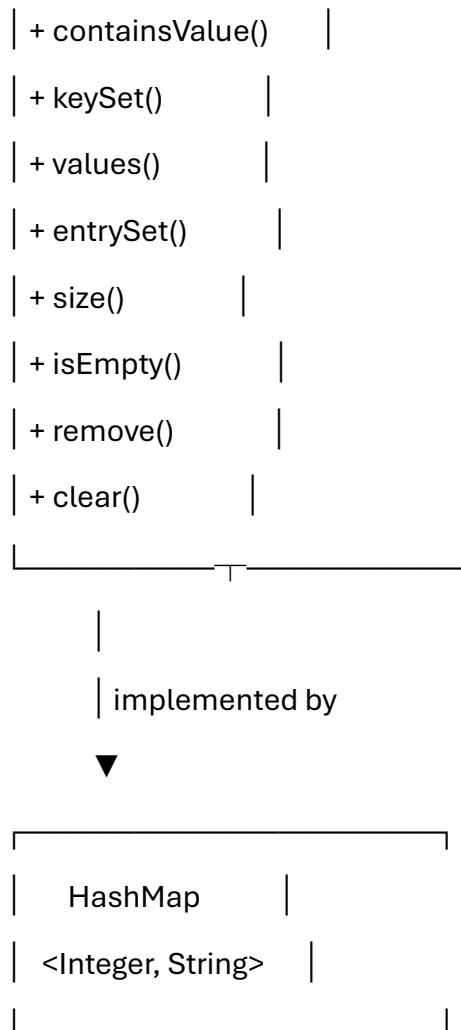
Algorithm

1. Start.
2. Create an empty HashMap named map.
3. Insert 101-Alice, 102-Bob, and 103-Charlie into the map.

4. Display the map.
5. Use get(102) to retrieve Bob.
6. Use containsKey(101) to check whether key 101 exists.
7. Use containsValue("Bob") to check whether Bob exists as a value.
8. Display the keys, values, and key-value entries.
9. Find and display the size of the map.
10. Check and display whether the map is empty.
11. Remove the entry having key 103.
12. Display the updated map.
13. Clear all entries from the map.
14. Display the empty map.
15. Stop.

UML Diagram





Code :

```
import java.util.HashMap;
import java.util.Map;

public class MapDemo {

    public static void main(String[] args) {

        // Create a Map using HashMap

        Map<Integer, String> map = new HashMap<>();

        map.put(101, "Alice");
        map.put(102, "Bob");
        map.put(103, "Charlie");
```

```

System.out.println("Map: " + map);

System.out.println("Get key 102: " + map.get(102));

System.out.println("Contains key 101: " + map.containsKey(101));

System.out.println("Contains value Bob: " + map.containsValue("Bob"));


// Display keys, values, and key-value entries

System.out.println("Key Set: " + map.keySet());

System.out.println("Values: " + map.values());

System.out.println("Entry Set: " + map.entrySet());


System.out.println("Size: " + map.size());

System.out.println("Is Empty: " + map.isEmpty());


map.remove(103);

System.out.println("After remove(103): " + map);


map.clear();

System.out.println("After clear(): " + map);

}

}

```

Output :

```

19
problems 8 Output Debug Console Terminal Ports
PS C:\Users\yabhi\abhi.java> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\yabhi\AppData\Roaming\Code\User\workspaceStorage\368729c680f0f32712f9803f080a0563\redhat.java\jdt_ws\abhi.java_92e0559b\bin' 'MapDemo'
Map: {101=Alice, 102=Bob, 103=Charlie}
Get key 102: Bob
Contains key 101: true
Contains value Bob: true
Key Set: [101, 102, 103]
Values: [Alice, Bob, Charlie]
Entry Set: [101=Alice, 102=Bob, 103=Charlie]
Size: 3
Is Empty: false
After remove(103): {101=Alice, 102=Bob}
After clear(): {}
PS C:\Users\yabhi\abhi.java>
Build failed, do you want to continue?

```


9. Title : Demonstration of Important Methods of SortedMap Interface in Java .

Aim / Objective : To demonstrate the important methods of the “SortedMap” interface.

System Requirements :

Hardware: Computer/Laptop, minimum 4 GB RAM.

Software: JDK, JRE, Java IDE, and operating system.

Steps – SortedMap

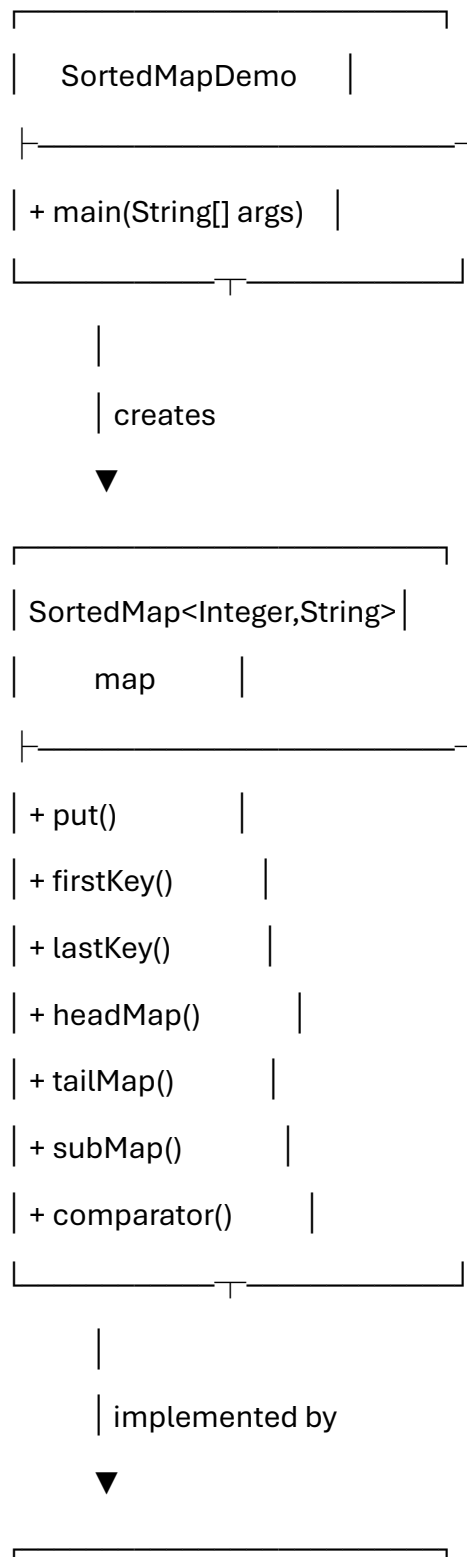
1. Start the program.
2. Import SortedMap and TreeMap from java.util.
3. Create a SortedMap<Integer, String> using TreeMap.
4. Add key-value pairs 101-Alice, 103-Charlie, 102-Bob, and 104-David.
5. Display the sorted map.
6. Use firstKey() to display the smallest key.
7. Use lastKey() to display the largest key.
8. Use headMap(103) to display entries with keys less than 103.
9. Use tailMap(102) to display entries with keys greater than or equal to 102.
10. Use subMap(102, 104) to display entries from key 102 up to, but not including, 104.
11. Use comparator() to display the comparator used for sorting.
12. Stop the program.

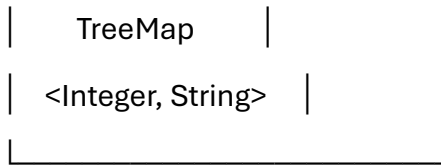
Algorithm

1. Start.
2. Create an empty TreeMap and store it in a SortedMap reference.
3. Insert the four key-value pairs into the map.
4. Display the map in sorted order of keys.
5. Find and display the first key using firstKey().
6. Find and display the last key using lastKey().
7. Display the portion before key 103 using headMap().
8. Display the portion from key 102 using tailMap().

9. Display the portion from key 102 to 104 using subMap().
10. Display the comparator using comparator().
11. Stop.

UML Diagram





Code :

```
import java.util.SortedMap;
import java.util.TreeMap;

public class SortedMapDemo {
    public static void main(String[] args) {

        // TreeMap maintains keys in sorted order
        SortedMap<Integer, String> map = new TreeMap<>();
        map.put(101, "Alice");
        map.put(103, "Charlie");
        map.put(102, "Bob");
        map.put(104, "David");

        System.out.println("Sorted Map: " + map);

        System.out.println("First Key: " + map.firstKey());
        System.out.println("Last Key: " + map.lastKey());

        // Display different portions of the sorted map
        System.out.println("Head Map: " + map.headMap(103));
        System.out.println("Tail Map: " + map.tailMap(102));
        System.out.println("Sub Map: " + map.subMap(102, 104));
    }
}
```

```

        System.out.println("Comparator: " + map.comparator());
    }
}

```

Output :

```

PS C:\Users\yabhi\abhi.java> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetails'
ges' '-cp' 'C:\Users\yabhi\AppData\Roaming\Code\User\workspaceStorage\368729c680f0f32712f9803f080a0563\redhat.java\jdt_ws
559b\bin' 'SortedMapDemo'
Sorted Map: {101=Alice, 102=Bob, 103=Charlie, 104=David}
First Key: 101
Last Key: 104
Head Map: {101=Alice, 102=Bob}
Tail Map: {102=Bob, 103=Charlie, 104=David}
Sub Map: {102=Bob, 103=Charlie}
Comparator: null
PS C:\Users\yabhi\abhi.java>

```

10 .Title :

Demonstration of Important Methods of NavigableMap Interface in Java.

Aim / Objective : To demonstrate the navigation and entry-manipulation methods of the “NavigableMap” interface.

System Requirements :

Hardware: Computer/Laptop, minimum 4 GB RAM.

Software: JDK, JRE, Java IDE, and operating system.

Steps – NavigableMap :

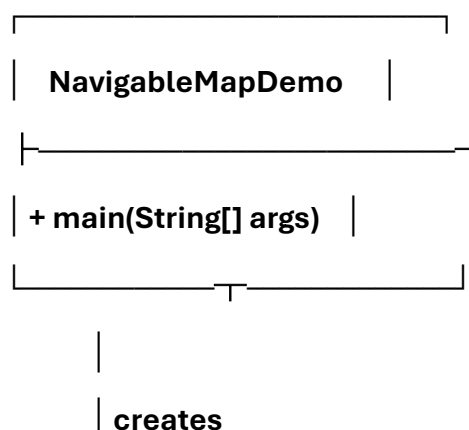
1. Start the program.
2. Import NavigableMap, TreeMap, and Map from java.util.
3. Create a NavigableMap<Integer, String> using TreeMap.
4. Add the key-value pairs 10-A, 30-B, 50-C, and 80-D.
5. Display the map.
6. Use lowerKey(30) to find the greatest key less than 30.
7. Use floorKey(30) to find the greatest key less than or equal to 30.
8. Use ceilingKey(25) to find the smallest key greater than or equal to 25.
9. Use higherKey(30) to find the smallest key greater than 30.
10. Display the first entry using firstEntry().
11. Display the last entry using lastEntry().

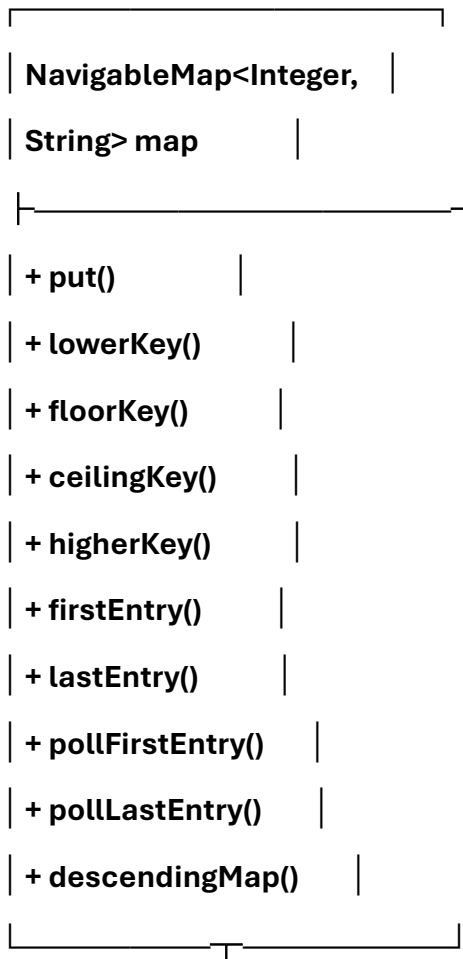
12. Remove and display the first entry using pollFirstEntry().
13. Remove and display the last entry using pollLastEntry().
14. Display the updated map.
15. Display the map in descending order using descendingMap().
16. Stop the program.

Algorithm :

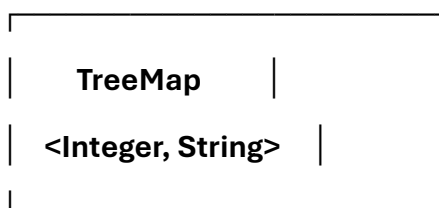
1. Start.
2. Create an empty TreeMap and store it in a NavigableMap reference.
3. Insert the four key-value pairs into the map.
4. Display the map in ascending key order.
5. Find the lower key of 30.
6. Find the floor key of 30.
7. Find the ceiling key of 25.
8. Find the higher key of 30.
9. Find and display the first and last entries.
10. Remove the first entry using pollFirstEntry().
11. Remove the last entry using pollLastEntry().
12. Display the remaining map.
13. Display the map in descending order using descendingMap().
14. Stop.

UML Diagram :





implemented by



Code :

```
import java.util.NavigableMap;
import java.util.TreeMap;
import java.util.Map;
```

```
public class NavigableMapDemo {  
    public static void main(String[] args) {  
  
        // Create a NavigableMap using TreeMap  
        NavigableMap<Integer, String> map = new TreeMap<>();  
  
        map.put(10, "A");  
        map.put(30, "B");  
        map.put(50, "C");  
        map.put(80, "D");  
  
        System.out.println("Map: " + map);  
  
        System.out.println("Lower Key: " + map.lowerKey(30));  
        System.out.println("Floor Key: " + map.floorKey(30));  
        System.out.println("Ceiling Key: " + map.ceilingKey(25));  
        System.out.println("Higher Key: " + map.higherKey(30));  
  
        System.out.println("First Entry: " + map.firstEntry());  
        System.out.println("Last Entry: " + map.lastEntry());  
  
        // Remove the first and last entries  
        System.out.println("Poll First Entry: " + map.pollFirstEntry());  
        System.out.println("Poll Last Entry: " + map.pollLastEntry());  
  
        System.out.println("Map after polling: " + map);  
  
        System.out.println("Descending Map: " + map.descendingMap());  
    }  
}
```

```
}  
  
}
```

Output :



```
Problems 8  Output  Debug Console  Terminal  Ports  
PS C:\Users\yabhi\abhi.java> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-cp' 'C:\Users\yabhi\AppData\Roaming\Code\User\workspaceStorage\368729c680f0f32712f9803559b\bin' 'NavigableMapDemo'  
Map: {10=A, 30=B, 50=C, 80=D}  
Lower Key: 10  
Floor Key: 30  
Ceiling Key: 30  
Higher Key: 50  
First Entry: 10=A  
Last Entry: 80=D  
Poll First Entry: 10=A  
Poll Last Entry: 80=D  
Map after polling: {30=B, 50=C}  
Descending Map: {50=C, 30=B}  
PS C:\Users\yabhi\abhi.java>
```

11. Title :

Demonstration of Important Methods of Iterator Interface in Java .

Aim / Objective .: To demonstrate the important methods of the “Iterator” interface for traversing and modifying a collection.

System Requirements :

Hardware: Computer/Laptop, minimum 4 GB RAM.

Software: JDK, JRE, Java IDE, and operating system.

Steps – SortedMap

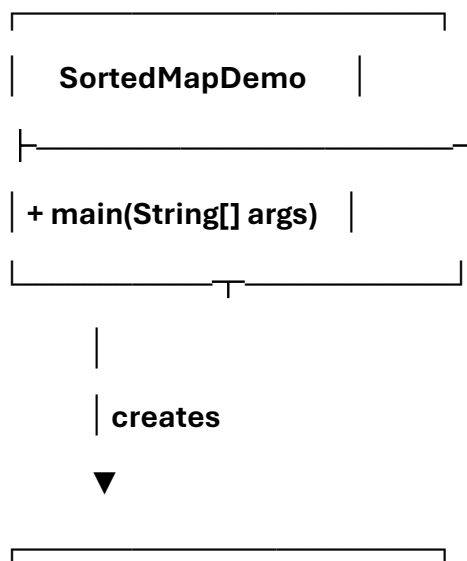
1. Start the program.
2. Import SortedMap and TreeMap from java.util.
3. Create a SortedMap<Integer, String> using TreeMap.
4. Add key-value pairs 101-Alice, 103-Charlie, 102-Bob, and 104-David.
5. Display the sorted map.
6. Use firstKey() to display the smallest key.
7. Use lastKey() to display the largest key.

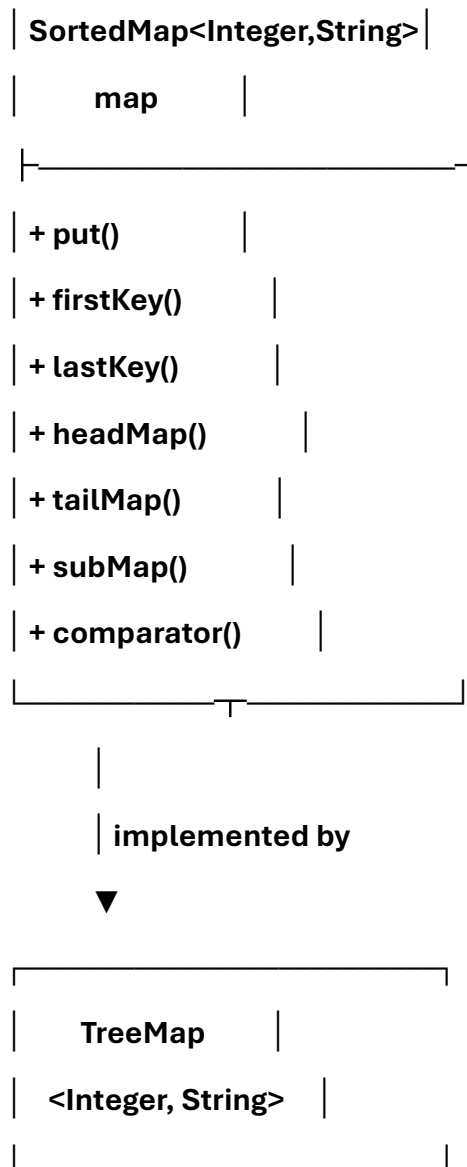
8. Use `headMap(103)` to display entries with keys less than 103.
9. Use `tailMap(102)` to display entries with keys greater than or equal to 102.
10. Use `subMap(102, 104)` to display entries from key 102 up to, but not including, 104.
11. Use `comparator()` to display the comparator used for sorting.
12. Stop the program.

Algorithm

1. Start.
2. Create an empty `TreeMap` and store it in a `SortedMap` reference.
3. Insert the four key-value pairs into the map.
4. Display the map in sorted order of keys.
5. Find and display the first key using `firstKey()`.
6. Find and display the last key using `lastKey()`.
7. Display the portion before key 103 using `headMap()`.
8. Display the portion from key 102 using `tailMap()`.
9. Display the portion from key 102 to 104 using `subMap()`.
10. Display the comparator using `comparator()`.
11. Stop.

UML Diagram





Important Methods

Method	Purpose
put()	Adds a key-value pair
firstKey()	Returns the smallest key
lastKey()	Returns the largest key
headMap(103)	Returns keys less than 103
tailMap(102)	Returns keys greater than or equal to 102
subMap(102,104)	Returns keys from 102 to less than 104

Method	Purpose
<code>comparator()</code>	Returns the comparator used for ordering

Important: `TreeMap` automatically maintains the keys in ascending sorted order when natural ordering is used. `comparator()` returns null here because the natural ordering of Integer keys is being used.

```
import java.util.NavigableMap; import java.util.TreeMap; import java.util.Map;
public class NavigableMapDemo { public static void main(String[] args) { // Create a
NavigableMap using TreeMap NavigableMap<Integer, String> map = new
TreeMap<>(); map.put(10, "A"); map.put(30, "B"); map.put(50, "C"); map.put(80,
"D"); System.out.println("Map: " + map); System.out.println("Lower Key: " +
map.lowerKey(30)); System.out.println("Floor Key: " + map.floorKey(30));
System.out.println("Ceiling Key: " + map.ceilingKey(25));
System.out.println("Higher Key: " + map.higherKey(30)); System.out.println("First
Entry: " + map.firstEntry()); System.out.println("Last Entry: " + map.lastEntry()); //
Remove the first and last entries System.out.println("Poll First Entry: " +
map.pollFirstEntry()); System.out.println("Poll Last Entry: " + map.pollLastEntry());
System.out.println("Map after polling: " + map); System.out.println("Descending
Map: " + map.descendingMap()); } }
```

Steps – NavigableMap

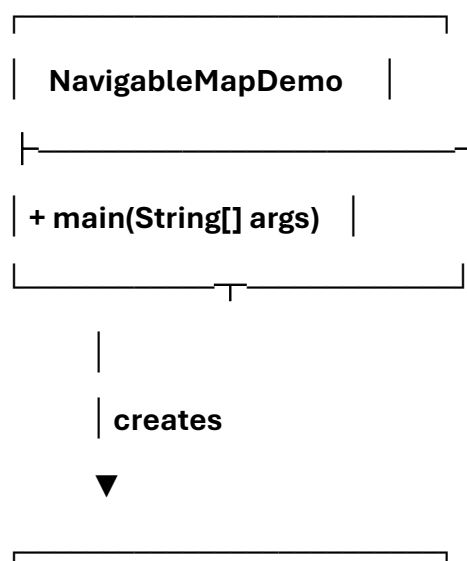
1. Start the program.
2. Import `NavigableMap`, `TreeMap`, and `Map` from `java.util`.
3. Create a `NavigableMap<Integer, String>` using `TreeMap`.
4. Add the key-value pairs 10-A, 30-B, 50-C, and 80-D.
5. Display the map.
6. Use `lowerKey(30)` to find the greatest key less than 30.
7. Use `floorKey(30)` to find the greatest key less than or equal to 30.
8. Use `ceilingKey(25)` to find the smallest key greater than or equal to 25.
9. Use `higherKey(30)` to find the smallest key greater than 30.
10. Display the first entry using `firstEntry()`.
11. Display the last entry using `lastEntry()`.
12. Remove and display the first entry using `pollFirstEntry()`.
13. Remove and display the last entry using `pollLastEntry()`.

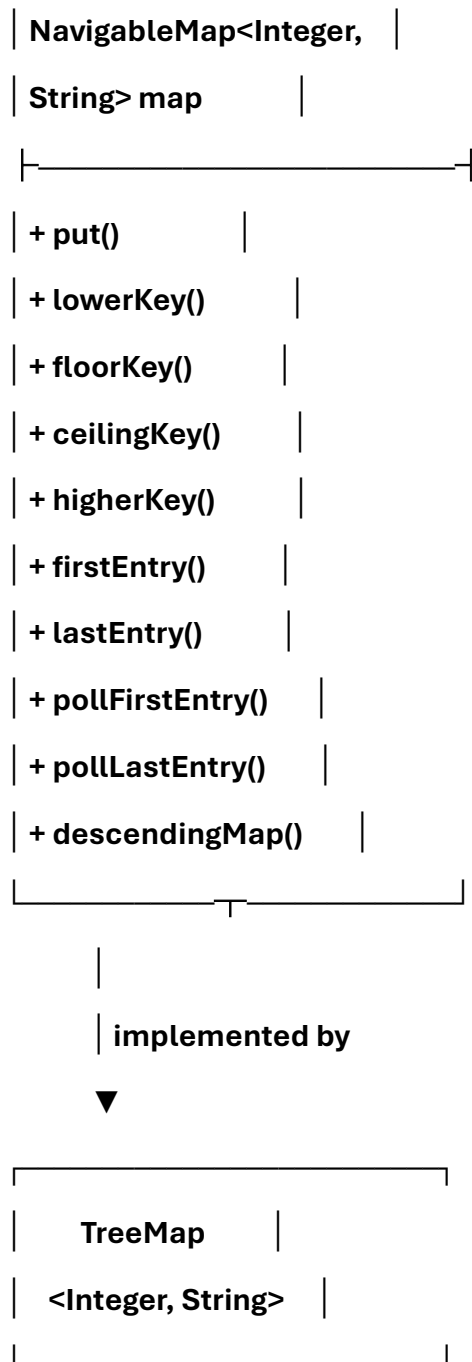
14. Display the updated map.
15. Display the map in descending order using `descendingMap()`.
16. Stop the program.

Algorithm

1. Start.
2. Create an empty `TreeMap` and store it in a `NavigableMap` reference.
3. Insert the four key-value pairs into the map.
4. Display the map in ascending key order.
5. Find the lower key of 30.
6. Find the floor key of 30.
7. Find the ceiling key of 25.
8. Find the higher key of 30.
9. Find and display the first and last entries.
10. Remove the first entry using `pollFirstEntry()`.
11. Remove the last entry using `pollLastEntry()`.
12. Display the remaining map.
13. Display the map in descending order using `descendingMap()`.
14. Stop.

UML Diagram





Important Methods

Method	Purpose
<code>put()</code>	Adds a key-value pair
<code>lowerKey(30)</code>	Greatest key less than 30
<code>floorKey(30)</code>	Greatest key less than or equal to 30
<code>ceilingKey(25)</code>	Smallest key greater than or equal to 25

Method	Purpose
<code>higherKey(30)</code>	Smallest key greater than 30
<code>firstEntry()</code>	Returns the first key-value entry
<code>lastEntry()</code>	Returns the last key-value entry
<code>pollFirstEntry()</code>	Removes and returns the first entry
<code>pollLastEntry()</code>	Removes and returns the last entry
<code>descendingMap()</code>	Returns the map in descending key order

Note: `TreeMap` automatically maintains the keys in ascending order, while `descendingMap()` provides a view in reverse order.

```
import java.util.ArrayList; import java.util.Iterator; import java.util.List; public class
IteratorDemo { public static void main(String[] args) { // Create an ArrayList for
iteration List<Integer> list = new ArrayList<>(); list.add(10); list.add(20); list.add(30);
list.add(40); Iterator<Integer> itr = list.iterator(); System.out.println("Elements:");
while (itr.hasNext()) { int value = itr.next(); System.out.println(value); // Remove 20
safely using the iterator if (value == 20) { itr.remove(); } } System.out.println("After
remove(): " + list); System.out.println("Using forEachRemaining()");
Iterator<Integer> itr2 = list.iterator(); itr2.forEachRemaining( value ->
System.out.println(value) ); } }
```

Steps – Iterator

1. Start the program.
2. Import `ArrayList`, `Iterator`, and `List` from `java.util`.
3. Create an `ArrayList` using the `List<Integer>` reference.
4. Add 10, 20, 30, and 40 to the list.
5. Create an iterator using `list.iterator()`.
6. Check whether the iterator has a next element using `hasNext()`.
7. Get each element using `next()` and display it.
8. When the value is 20, remove it safely using `itr.remove()`.
9. Continue until all elements are visited.
10. Display the list after removing 20.
11. Create another iterator `itr2`.

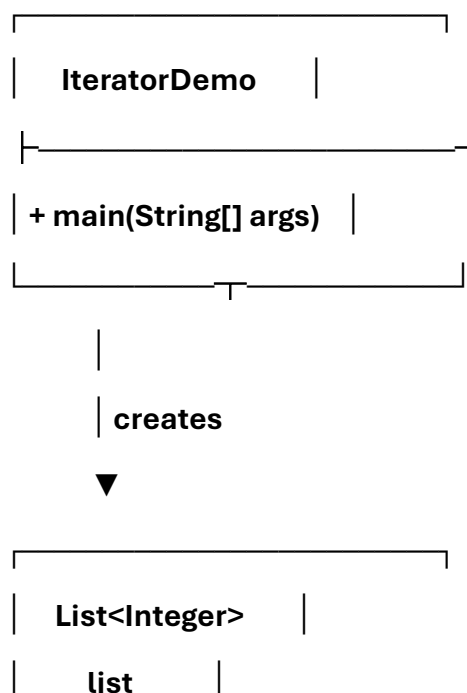
12. Use `forEachRemaining()` to display all remaining elements.

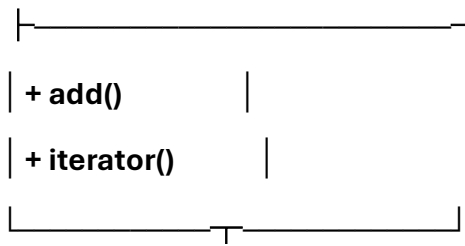
13. Stop the program.

Algorithm

1. Start.
2. Create an empty `ArrayList`.
3. Insert 10, 20, 30, and 40 into the list.
4. Create an iterator for the list.
5. Repeat while `hasNext()` returns true.
6. Get the next element using `next()`.
7. Display the element.
8. If the element is 20, remove it using `remove()`.
9. Continue until the iterator reaches the end of the list.
10. Display the updated list.
11. Create a new iterator for the updated list.
12. Use `forEachRemaining()` to display every remaining element.
13. Stop.

UML Diagram

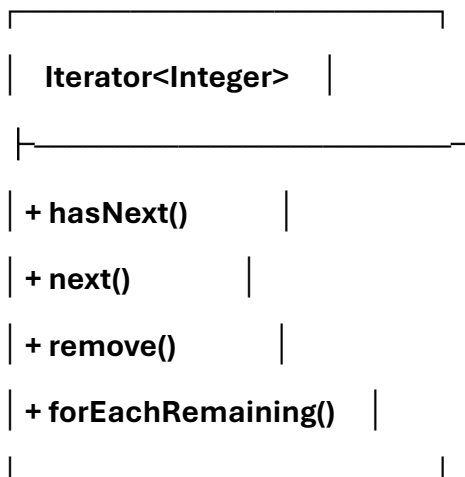




| implemented by
▼



| iterator()
▼



Code :

```
import java.util.ArrayList;

import java.util.Iterator;

import java.util.List;

public class IteratorDemo {
```



```
public static void main(String[] args) {

    // Create an ArrayList for iteration
    List<Integer> list = new ArrayList<>();

    list.add(10);
    list.add(20);
    list.add(30);
    list.add(40);

    Iterator<Integer> itr = list.iterator();

    System.out.println("Elements:");

    while (itr.hasNext()) {

        int value = itr.next();
        System.out.println(value);

        // Remove 20 safely using the iterator
        if (value == 20) {
            itr.remove();
        }
    }

    System.out.println("After remove(): " + list);

    System.out.println("Using forEachRemaining():");
```

```

Iterator<Integer> itr2 = list.iterator();

itr2.forEachRemaining(
    value -> System.out.println(value)
);
}
}

```

Output :

```

Problems 8  Output  Debug Console  Terminal  Ports
PS C:\Users\yabhi\abhi.java> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-p
ges' '-cp' 'C:\Users\yabhi\AppData\Roaming\Code\User\workspaceStorage\368729c680f0f32712f98
559b\bin' 'IteratorDemo'
Elements:
10
20
30
40
After remove(): [10, 30, 40]
Using forEachRemaining():
10
30
40
PS C:\Users\yabhi\abhi.java>

```

12. **Title :** Demonstration of Important Methods of ListIterator Interface in Java. **Aim**
/ Objective : To demonstrate forward and backward traversal and modification
 methods of the “ListIterator” interface.

System Requirements :

Hardware: Computer/Laptop, minimum 4 GB RAM.

Software: JDK, JRE, Java IDE, and operating system.

Steps – ListIterator

1. **Start the program.**

2. Import ArrayList, List, and ListIterator from java.util.
3. Create an ArrayList using the List<Integer> reference.
4. Add 10, 20, and 30 to the list.
5. Create a ListIterator using list.listIterator().
6. Display the initial list.
7. Perform forward traversal using hasNext() and next().
8. Display each element, its next index, and previous index.
9. Perform backward traversal using hasPrevious() and previous().
10. Move the iterator to the required position using next().
11. Modify the element using set(15).
12. Display the updated list.
13. Insert 12 using add().
14. Display the updated list.
15. Move to the next element and remove it using remove().
16. Display the final list.
17. Stop the program.

Algorithm

1. Start.
2. Create an empty ArrayList.
3. Insert 10, 20, and 30 into the list.
4. Create a ListIterator for the list.
5. Display the initial list.
6. Traverse the list in the forward direction using hasNext() and next().
7. Display each element, next index, and previous index.
8. Traverse the list backward using hasPrevious() and previous().
9. Move the iterator to the required position.
10. Change the current element to 15 using set().
11. Insert 12 using add().

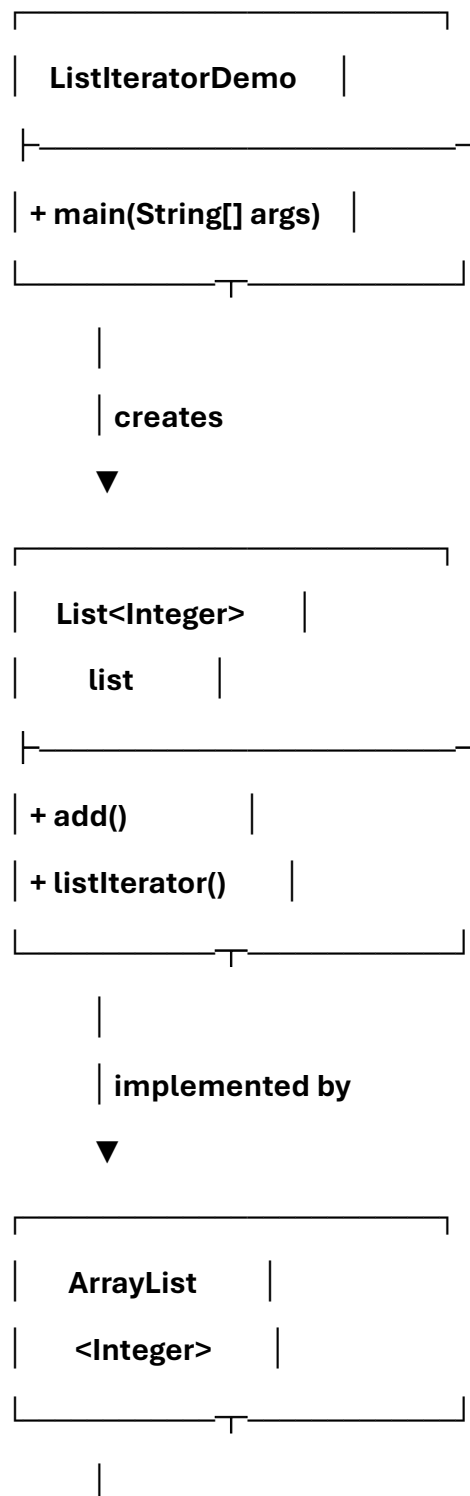
12. Move to the next element using next().

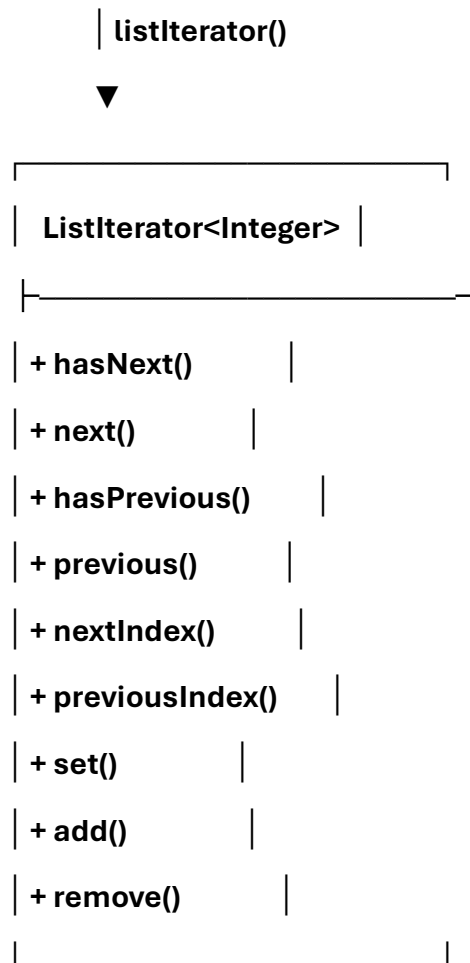
13. Remove the selected element using remove().

14. Display the final list.

15. Stop.

UML Diagram





Code :

```
import java.util.ArrayList;

import java.util.List;

import java.util.ListIterator;

public class ListIteratorDemo {

    public static void main(String[] args) {

        // Create a list for forward and backward traversal

        List<Integer> list = new ArrayList<>();

        list.add(10);

        list.add(20);
```

```
list.add(30);
```

```
ListIterator<Integer> itr = list.listIterator();
```

```
System.out.println("Initial List: " + list);
```

```
System.out.println("\nForward Traversal:");
```

```
while (itr.hasNext()) {
```

```
    System.out.println("Element: " + itr.next());
```

```
    System.out.println("Next Index: " + itr.nextIndex());
```

```
    System.out.println("Previous Index: " + itr.previousIndex());
```

```
}
```

```
System.out.println("\nBackward Traversal:");
```

```
while (itr.hasPrevious()) {
```

```
    System.out.println(itr.previous());
```

```
}
```

```
// Modify and insert elements using ListIterator
```

```
itr.next();
```

```
itr.set(15);
```

```
System.out.println("\nAfter set(): " + list);
```

```
itr.add(12);
```

```
System.out.println("After add(): " + list);
```

```
itr.next();
```

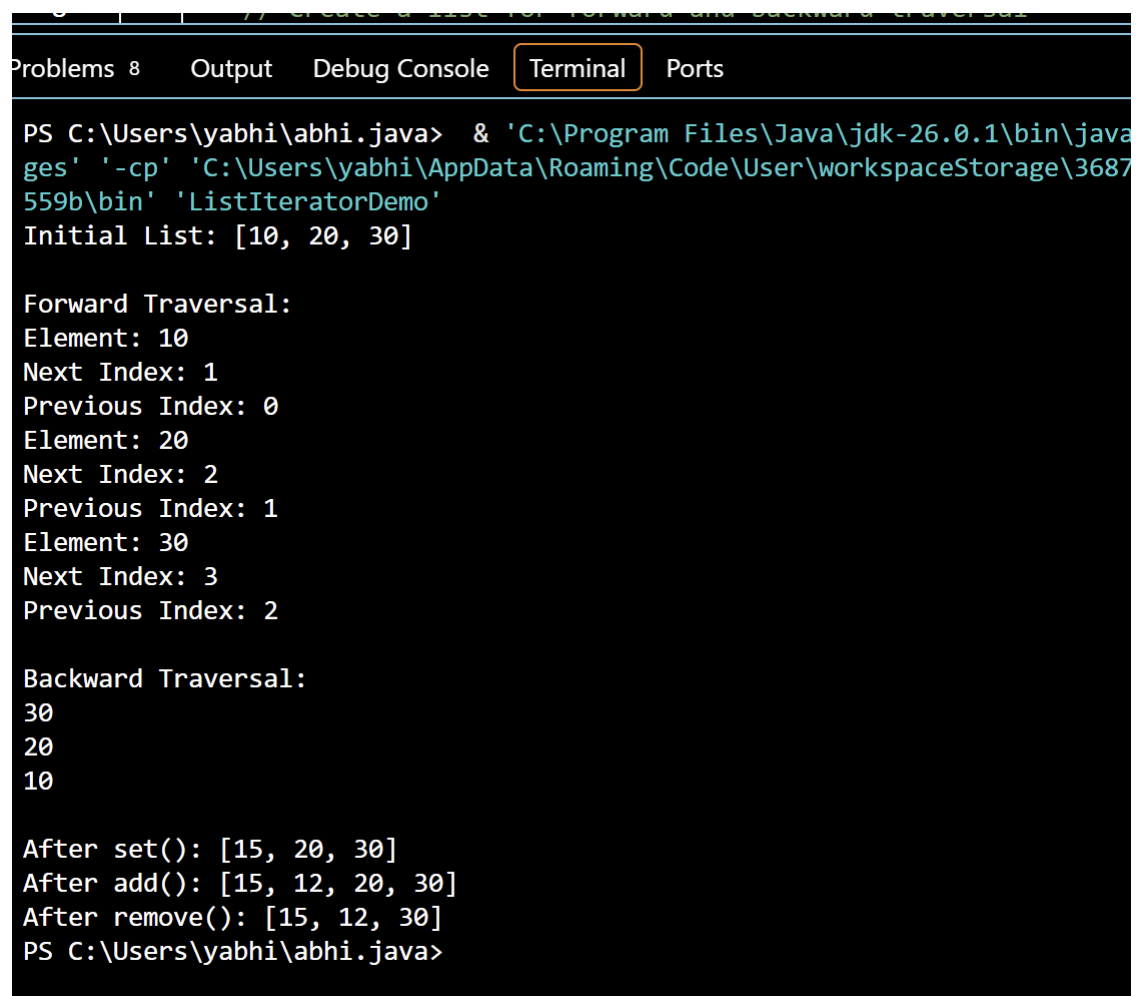
```
itr.remove();
```

```
System.out.println("After remove(): " + list);
```

```
}
```

```
}
```

Output :



```
PS C:\Users\yabhi\abhi.java> & 'C:\Program Files\Java\jdk-26.0.1\bin\java
ges' '-cp' 'C:\Users\yabhi\AppData\Roaming\Code\User\workspaceStorage\3687
559b\bin' 'ListIteratorDemo'
Initial List: [10, 20, 30]

Forward Traversal:
Element: 10
Next Index: 1
Previous Index: 0
Element: 20
Next Index: 2
Previous Index: 1
Element: 30
Next Index: 3
Previous Index: 2

Backward Traversal:
30
20
10

After set(): [15, 20, 30]
After add(): [15, 12, 20, 30]
After remove(): [15, 12, 30]
PS C:\Users\yabhi\abhi.java>
```